



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное
учреждение высшего
образования
«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
по дисциплине «Разработка баз данных»

Студент группы

ИКБО-50-23. Астахов С.П..

(подпись)

Преподаватель

Мажей Я.В

(подпись)

Москва 2025 г.

Содержание

1. Цель работы и постановка задач:.....	3
2. Ход выполнения работы.....	5
2.1. Бизнес-правила и ограничения целостности.....	5
2.2. Создание всех таблиц, реализовав все описанные ограничения целостности.....	6
2.3. Заполнение созданных таблиц согласованными тестовыми данными.	8
2.4. Выполнение 6 SQL-запросов к таблицам.	9
2.5. Выполнение по два SQL-запроса к таблицам для демонстрации работы предложений ORDER BY, GROUP BY и HAVING.	13
3. Вывод	14
4. Дополнительная литература	16

1. Цель работы и постановка задач:

Цель: Целью данной практической работы является формирование и закрепление у студентов фундаментальных навыков работы с реляционными базами данных на примере СУБД Postgres Pro.

Постановка задачи:

1. На основе логической модели данных, созданной в прошлом семестре, письменно описать не менее 5 различных бизнес-правил и не менее 3 ограничений целостности для таблиц. Выбор бизнес-правил и ограничений целостности производится на усмотрение студента. Результаты представить в виде таблиц.
2. С использованием DDL-оператора CREATE TABLE создать все необходимые таблицы (согласно созданной в прошлом семестре логической модели данных) в СУБД Postgres Pro, корректно реализовав все описанные ограничения целостности.
3. Заполнить созданные таблицы согласованными тестовыми данными (не менее 5-7 записей на таблицу, где это применимо) с помощью оператора INSERT INTO.
4. Составить и выполнить не менее 6 SQL-запросов к таблицам, иллюстрирующих использования различных операторов.

SELECT и WHERE. В составленных запросах должны быть использованы все приведённые примеры.

5. Составить и выполнить по два SQL-запроса к таблицам для демонстрации работы предложений ORDER BY, GROUP BY и HAVING.
6. Каждый SQL-запрос сопровождать комментарием, объясняющим его назначение и логику работы.

2. Ход выполнения работы

2.1. Бизнес-правила и ограничения целостности.

Название столбца	Тип данных	Ограничение	Обоснование (бизнес-правило)
id_game	SERIAL	PRIMARY KEY	Уникальный индификатор игры, генерируется автоматически
name	VARCHAR (350)	NOT NULL	Название игры, является обязательным и не может быть пустым
id_developer	INT	NOT NULL, FOREIGN KEY(developer), CHECK(id_developer > 0)	Ссылка на разработчика, не может быть отрицательной и должна быть заполнена обязательно
id_genres	INT	NOT NULL, FOREIGN KEY(genres), CHECK(id_genres > 0)	Ссылка на жанр, не может быть отрицательной и должна быть заполнена обязательно
release_date	DATE	NOT NULL	Дата релиза, не должна быть пустая
ranking	FLOAT	NOT NULL, CHECK (ranking >= 0 and ranking <= 10)	Оценка игры, не может быть пустой и должна быть в промежутке: 0..10

2.2. Создание всех таблиц, реализовав все описанные ограничения целостности.

Возьмем код с прошлого семестра и дополним ограничения, после создадим базу данных с новыми значениями.

```
CREATE TABLE _genres (  
    id_genres serial PRIMARY KEY, name  
    varchar(80) NOT NULL  
);  
CREATE TABLE _developers_game (  
    id_developers_game serial  
    PRIMARY KEY, name varchar(80) NOT NULL,  
    country varchar(80) NOT NULL  
);  
CREATE TABLE _platform (  
    id_platform serial PRIMARY KEY, name  
    varchar(50) NOT NULL, manufacturer  
    varchar(80) NOT NULL  
);  
CREATE TABLE _user (  
    id_user serial PRIMARY KEY, name  
    varchar(60) NOT NULL, surname  
    varchar(60) NOT NULL,  
    search_queries varchar(500) NOT NULL, email  
    varchar(80) NOT NULL, date_registration date NOT  
    NULL  
);  
CREATE TABLE _games (  
    id_game serial PRIMARY KEY, name  
    varchar(350) NOT NULL, id_developer  
    integer NOT NULL, id_genres integer NOT  
    NULL, release_date date NOT NULL,  
    ranking float8 NOT NULL,  
    CHECK(id_developer > 0),  
    CHECK(id_genres > 0),  
    CHECK(ranking >= 0 AND ranking <= 10)  
);  
CREATE TABLE _games_platform (  
    id_games_platform  
    serial PRIMARY KEY, id_games integer NOT  
    NULL, id_platform integer NOT NULL  
);  
CREATE TABLE _favorites (  
    id_favorites serial PRIMARY KEY, age_of_the_duplicate  
    date NOT NULL  
);  
CREATE TABLE _reviews (  
    id_reviews serial PRIMARY KEY, name  
    varchar(80) NOT NULL, review  
    text NOT NULL, rating  
    integer NOT NULL, date  
    date NOT NULL
```

```

        id_review serial PRIMARY KEY, id_user
        integer NOT NULL, id_game integer NOT
        NULL, ranking float8 NOT NULL,
        text_ranking text NOT NULL, date date
        NOT NULL
    );

CREATE TABLE user_favorites ( id_user_favorites serial
        PRIMARY KEY, id_user integer NOT NULL,
        id_favorites integer NOT NULL
    );

CREATE TABLE games_genres ( id_games_genres serial
        PRIMARY KEY, id_games integer NOT NULL,
        id_genres integer NOT NULL
    );

CREATE TABLE recommendations ( id_recommendations serial
PRIMARY KEY,
        id_game integer NOT NULL,
        id_user integer NOT NULL, reason
        text NOT NULL
    );

CREATE TABLE analysts (
        id_analyst serial PRIMARY KEY, name
        varchar(80) NOT NULL, role
        varchar(100) NOT NULL
    );

CREATE TABLE analysts_recommendations (
        id_analysts_recommendations serial PRIMARY KEY, id_analysts
        integer NOT NULL, id_recommendations integer NOT NULL
    );

CREATE TABLE favorites_games ( id_favorites_games serial
        PRIMARY KEY, id_favorites integer NOT NULL,
        id_games integer NOT NULL
    );

CREATE TABLE games_developers ( id_games_developers
        serial PRIMARY KEY, id_games integer NOT NULL,
        id_developers integer NOT NULL
    );

ALTER TABLE games ADD CONSTRAINT games_id_genres fk FOREIGN KEY (id_genres)
REFERENCES genres(id_genres); ALTER TABLE games ADD
CONSTRAINT games_id_developer fk FOREIGN KEY (id_developer) REFERENCES
developers_game(id_developers_game); ALTER TABLE games platform ADD CONSTRAINT
games_platform_id_games fk FOREIGN KEY (id_games) REFERENCES
games(id_game); ALTER TABLE games platform ADD CONSTRAINT games_platform_id_platform fk
FOREIGN KEY (id_platform) REFERENCES platform(id_platform); ALTER TABLE reviews
ADD CONSTRAINT reviews_id_game fk FOREIGN KEY (id_game)

```

```

REFERENCES games(id_game);
ALTER TABLE reviews ADD CONSTRAINT reviews_id_user_fk FOREIGN KEY (id_user) REFERENCES
user (id_user);
ALTER TABLE user_favorites ADD CONSTRAINT user_favorites_id_user_fk FOREIGN KEY(id_user)
REFERENCES user (id_user);
ALTER TABLE user_favorites ADD CONSTRAINT user_favorites_id_favorites_fk
FOREIGN KEY (id_favorites) REFERENCES favorites(id_favorites);
ALTER TABLE games_genres ADD CONSTRAINT games_genres_id_games_fk FOREIGN KEY(id_games)
REFERENCES games(id_game);
ALTER TABLE games_genres ADD CONSTRAINT games_genres_id_genres_fk FOREIGN
KEY(id_genres) REFERENCES genres(id_genres);
ALTER TABLE recommendations ADD CONSTRAINT recommendations_id_game_fk FOREIGN
KEY(id_game) REFERENCES games(id_game);
ALTER TABLE recommendations ADD CONSTRAINT recommendations_id_user_fk FOREIGN KEY(id_user)
REFERENCES user (id_user);
ALTER TABLE favorites_games ADD CONSTRAINT favorites_games_id_favorites_fk
foreign KEY (id_favorites) REFERENCES favorites(id_favorites);
ALTER TABLE favorites_games ADD CONSTRAINT favorites_games_id_games_fk
FOREIGN KEY(id_games) REFERENCES games(id_game);
ALTER TABLE games_developers ADD CONSTRAINT games_developers_id_games_fk
foreign KEY (id_games) REFERENCES games(id_game);
ALTER TABLE games_developers ADD CONSTRAINT games_developers_id_developers_fk FOREIGN KEY
(id_developers) REFERENCES developers_game(id_developers_game); ALTER TABLE
analysts_recommendations ADD CONSTRAINT analysts_recommendations_id_analysts_fk FOREIGN KEY
(id_analysts) REFERENCES analysts(id_analyst);
ALTER TABLE analysts_recommendations ADD CONSTRAINT
analysts_recommendations_id_recommendations_fk FOREIGN KEY(id_recommendations) REFERENCES
recommendations(id_recommendations);

```

2.3. Заполнение созданных таблиц согласованными тестовыми данными.

Заполним таблицу games данными. Для этого заполним еще родительские таблицы(ссылочные таблицы).

```

insert into
developers_game(name, country)
values
('Team Cherry', 'Австралия'),
('Valve Corporation', 'США');

insert into genres(name)

```



```

values
('Метроидвания'),
('Головоломки'),
('Стратегия'),
('Шутер');

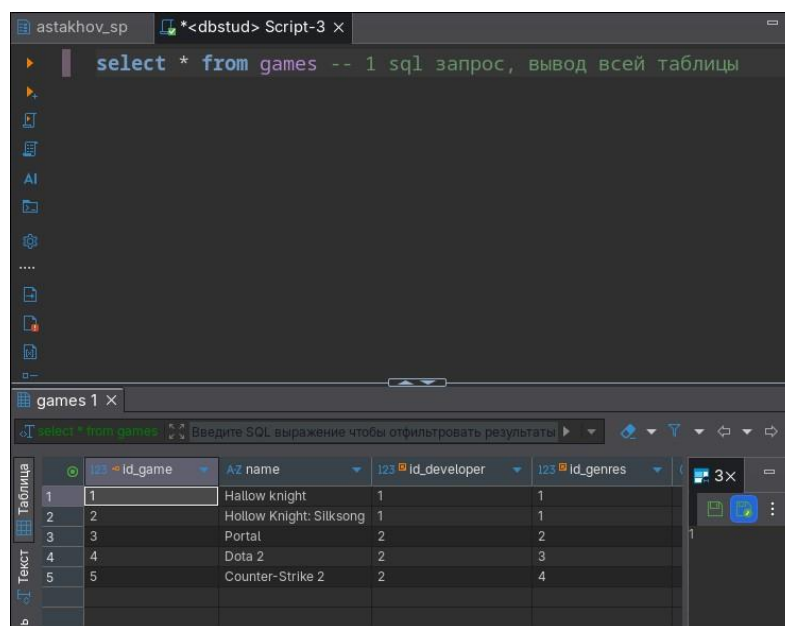
insert into
games(name,      id developer,      id genres,      release date,
ranking)
values
('Hallow knight', 1, 1, '2017-02-24', 8.2),
('Hollow Knight: Silksong', 1, 1, '2025-09-04', 9.1),
('Portal', 2, 2, '2007-10-10', 9.0),
('Dota 2', 2, 3, '2013-07-09', 7.2),
('Counter-Strike 2', 2, 4, '2023-09-27', 6.1);

```

Далее в пунктах 2.4-2.5 мы сделаем несколько sql запросов и опишем их комментариями (пункт 6)

2.4. Выполнение 6 SQL-запросов к таблицам.

I. Выбор всех столбцов (*)



The screenshot shows a database client interface with a script editor and a results pane. The script editor contains the query: `select * from games -- 1 sql запрос, вывод всей таблицы`. The results pane shows a table with 5 rows and 5 columns: `id_game`, `name`, `id_developer`, `id_genres`, and `release date`. The data is as follows:

	id_game	name	id_developer	id_genres	release date
1	1	Hallow knight	1	1	2017-02-24
2	2	Hollow Knight: Silksong	1	1	2025-09-04
3	3	Portal	2	2	2007-10-10
4	4	Dota 2	2	3	2013-07-09
5	5	Counter-Strike 2	2	4	2023-09-27

Рисунок 1 — Вывод всех столбцов

II. Выбор конкретных полей и использование псевдонима (AS)

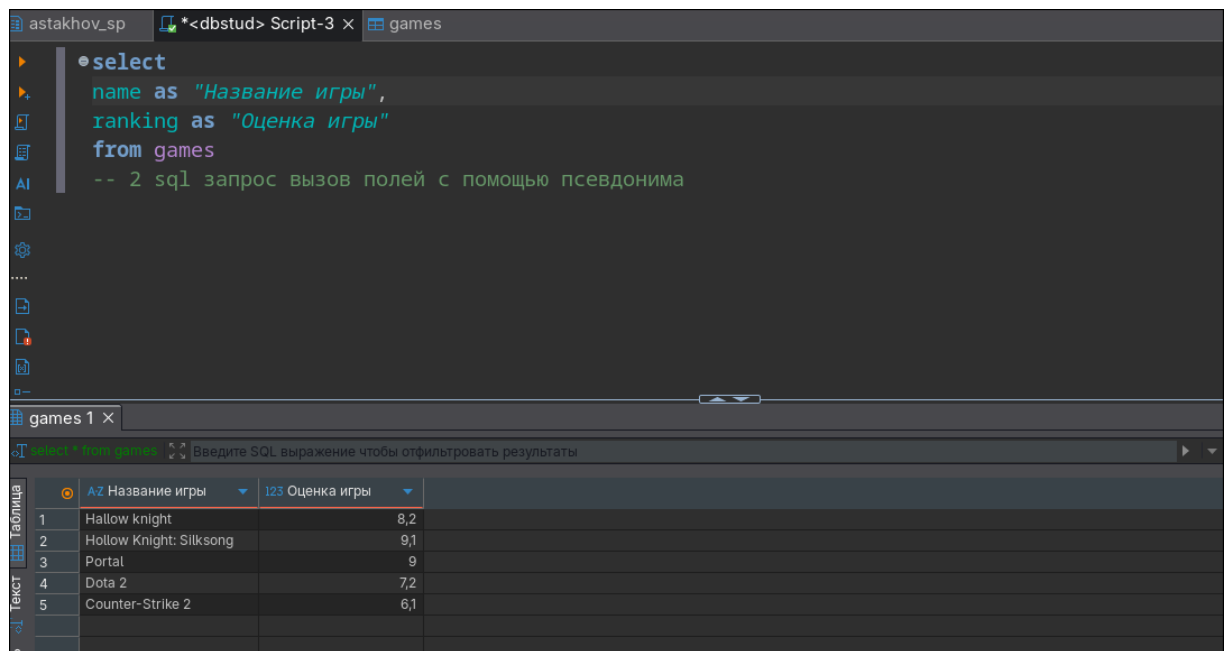


Рисунок 2 — Вывод столбцов с помощью псевдонима

III. Выражение в списке выборки

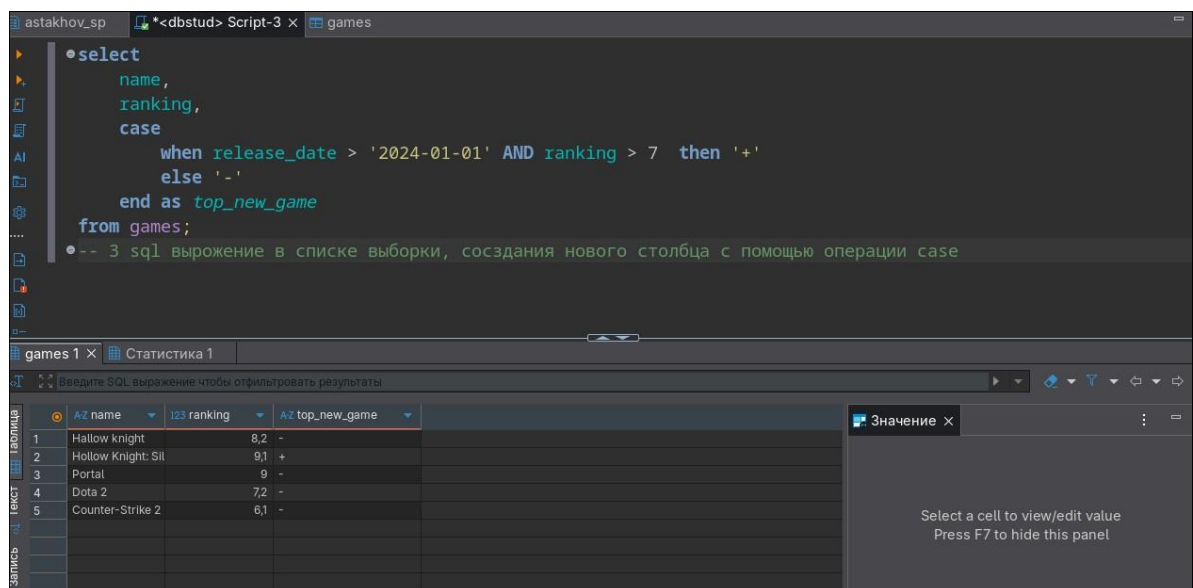


Рисунок 3 — Выражение в списке выборки

IV. Удаление дубликатов (DISTINCT)

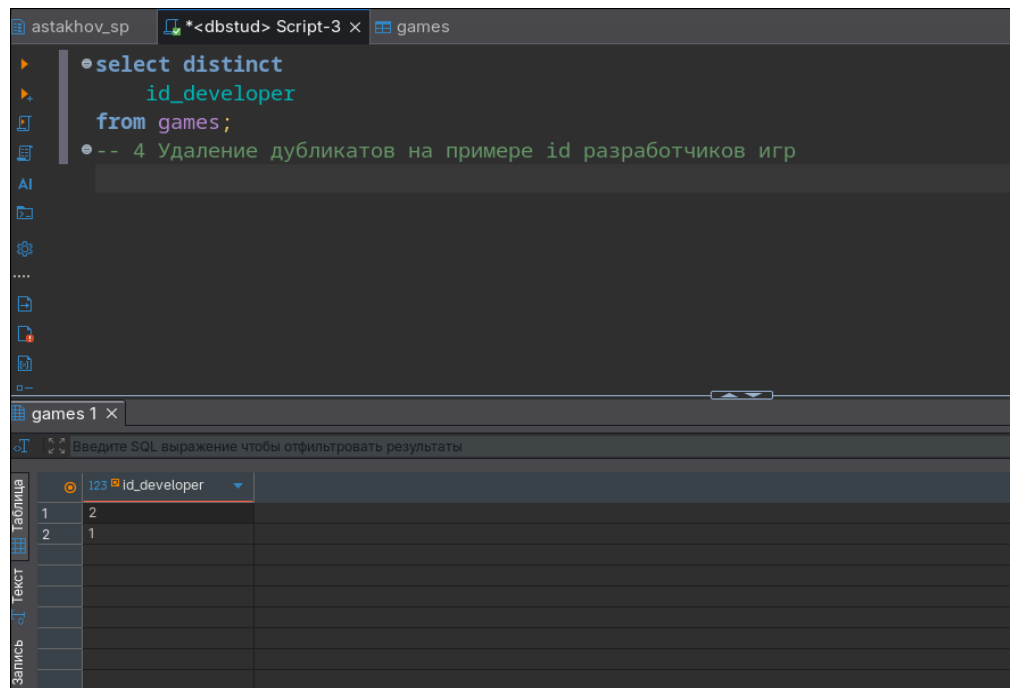


Рисунок 4 — Удаление дубликатов и вывод

V. Простое условие и логические связки (AND, OR)

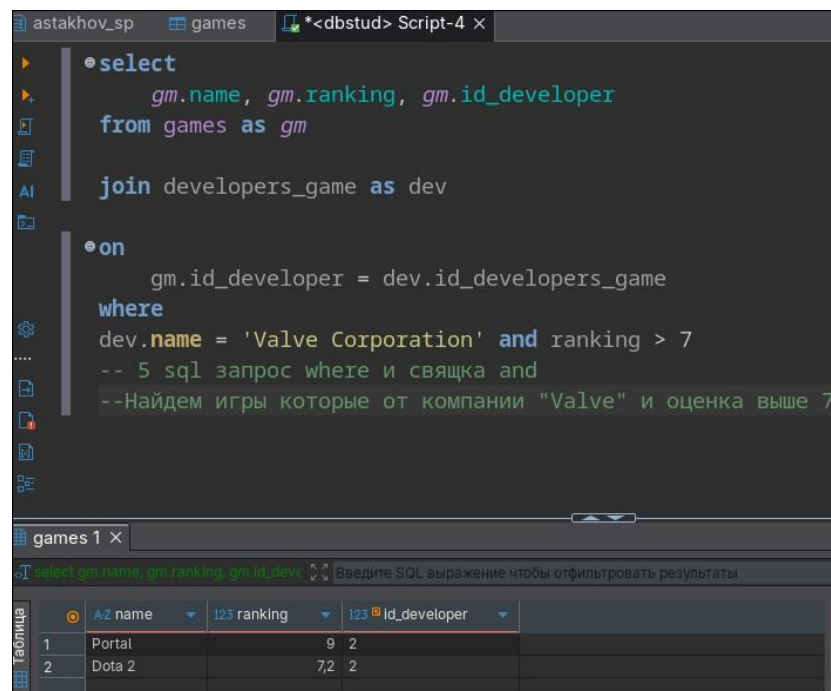


Рисунок 5 - Условие и логические связки

VI. Проверка принадлежности диапазону (BETWEEN)

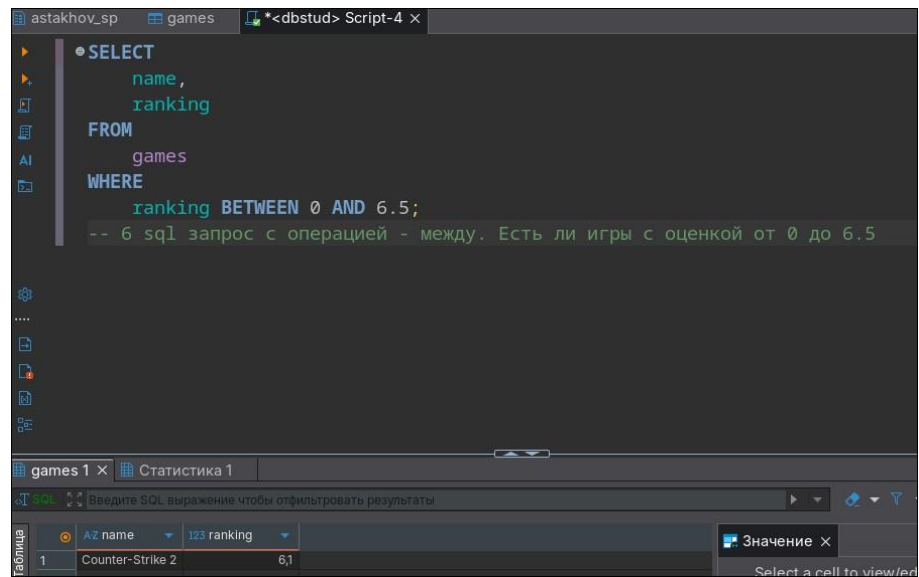
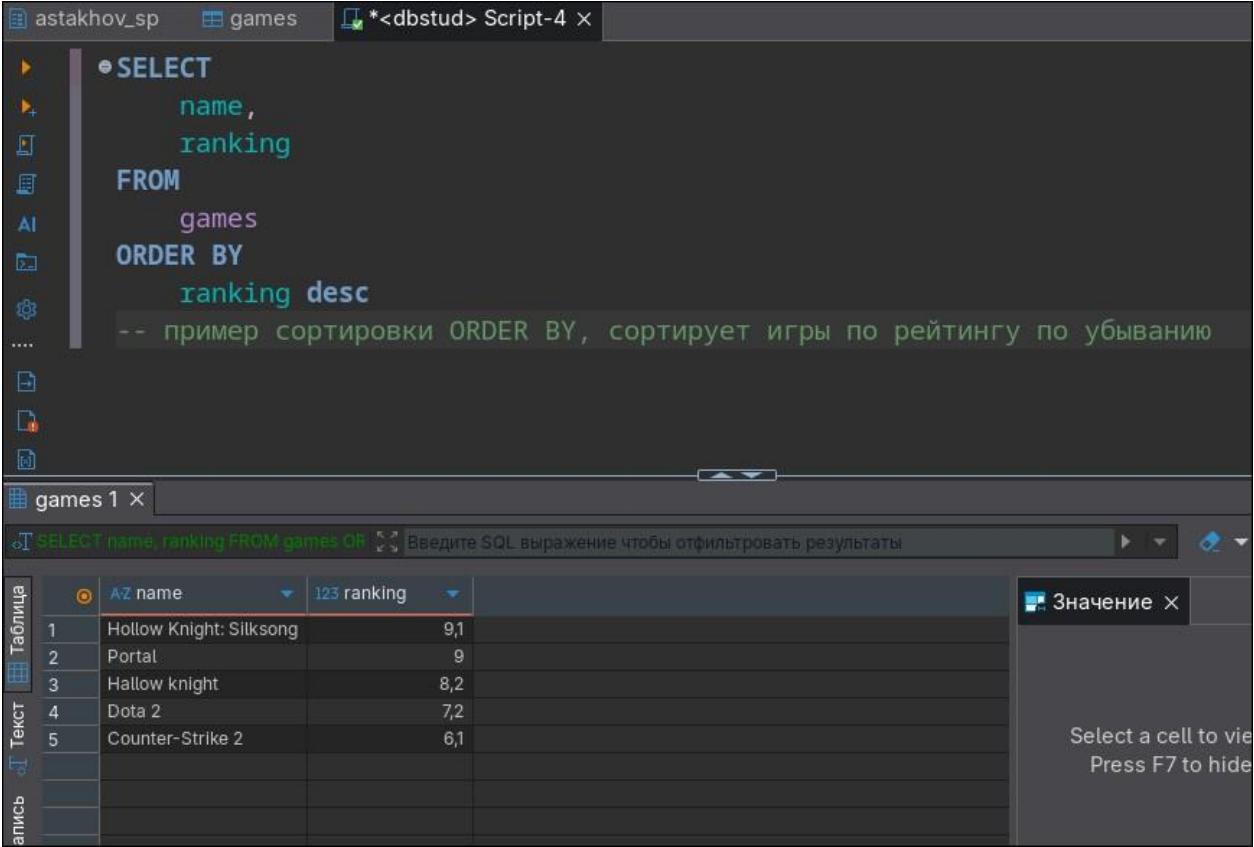


Рисунок 6 — Оценка игры с помощью «между»

2.5. Выполнение по два SQL-запроса к таблицам для демонстрации работы предложений **ORDER BY**, **GROUP BY** и **HAVING**.



The screenshot shows a database IDE with a SQL query editor and a results pane. The query is as follows:

```
SELECT
    name,
    ranking
FROM
    games
ORDER BY
    ranking desc
-- пример сортировки ORDER BY, сортирует игры по рейтингу по убыванию
```

The results pane displays a table with 5 rows and 2 columns: 'name' and 'ranking'. The data is sorted by ranking in descending order.

	AZ name	123 ranking
1	Hollow Knight: Silksong	9,1
2	Portal	9
3	Hallow knight	8,2
4	Dota 2	7,2
5	Counter-Strike 2	6,1

Рисунок 7 — Сортировка через **ORDER BY** по убыванию

```

SELECT
    name,
    release_date
FROM
    games
ORDER BY
    release_date asc;
-- пример сортировки ORDER BY, сортирует игры по возрастанию даты релиза

```

	Az name	release_date
1	Portal	2007-10-10
2	Dota 2	2013-07-09
3	Hallow knight	2017-02-24
4	Counter-Strike 2	2023-09-27
5	Hollow Knight: Sil	2025-09-04

Рисунок 8 - Сортировка через **ORDER BY** по возрастанию

```

-- Группируем игры по разработчикам и считаем статистику для каждого
SELECT
    d.name as developer, -- Название разработчика
    COUNT(*) as games_count, -- Сколько всего игр у разработчика
    AVG(g.rating) as avg_rating -- Средний рейтинг всех игр
FROM games g
JOIN developers_game d ON g.id_developer = d.id_developers_game -- Соединяем с таблицей разработчиков
GROUP BY d.name; -- Группируем результаты по имени разработчика

```

	Az developer	games_count	avg_rating
1	Team Cherry	2	8,65
2	Valve Corporation	3	7,433333333

Рисунок 9 — Сортировка по **GROUP BY**

astakhov_sp games *<dbstud> Script-4 x

-- Ищем разработчиков, у которых средний рейтинг игр выше 8.0

```
SELECT
    d.name as developer,
    COUNT(*) as games_count,
    AVG(g.rating) as avg_rating
FROM games g
JOIN developers_game d ON g.id_developer = d.id_developers_game
GROUP BY d.name -- Сначала группируем по разработчикам
HAVING AVG(g.rating) > 8.0; -- Затем оставляем только группы с средним рейтингом > 8.0
```

developers_game 1 x Статистика 1

SQL Введите SQL выражение чтобы отфильтровать результаты

	AZ developer	123 games_count	123 avg_rating	
1	Team Cherry	2	8,65	

Значение x

Select a cell to view/edit
Press F7 to hide this

Рисунок 10 — Сортировка heaving

3. Вывод

Закрепили фундаментальные навыки работы с реляционными базами данных на примере СУБД Postgres Pro. Изучили ограничения в базах данных, создали базу данных и изменили под ее таблицу, заполнили таблицы и выводили с помощью sql запросов. Также познакомились с такими командами как ORDER BY, GROUP BY и HAVING.

4. Дополнительная литература

1. Литература_Разработка_баз_данных_Новиков Б.А. Основы технологий баз данных: учебное пособие / Б.А. Новиков, Е.А. Горшкова, Н.Г. Графеева; под.ред. Е.В. Рогова. – 2-е изд. – М.: ДМК Пресс, 2020. – 582 с. Гиперссылка
2. Литература_Разработка_баз_данных_Моргунов Е.П. PostgreSQL. Основы языка SQL: учеб. Пособие / Е.П. Моргунов; под. ред. Е.В. Рогова, П.В. Лузанова. – СПб.: БХВ-Петербург, 2018. – 336 с.: ил.
3. Путеводитель по базам данных. — М.: ДМК-Пресс, 2024. — 520 с. ISBN 978-5-93700-287-7